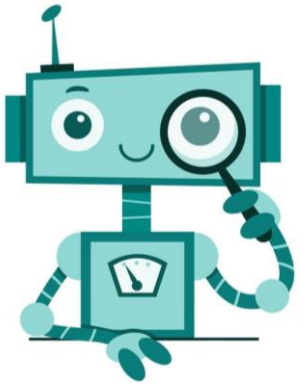
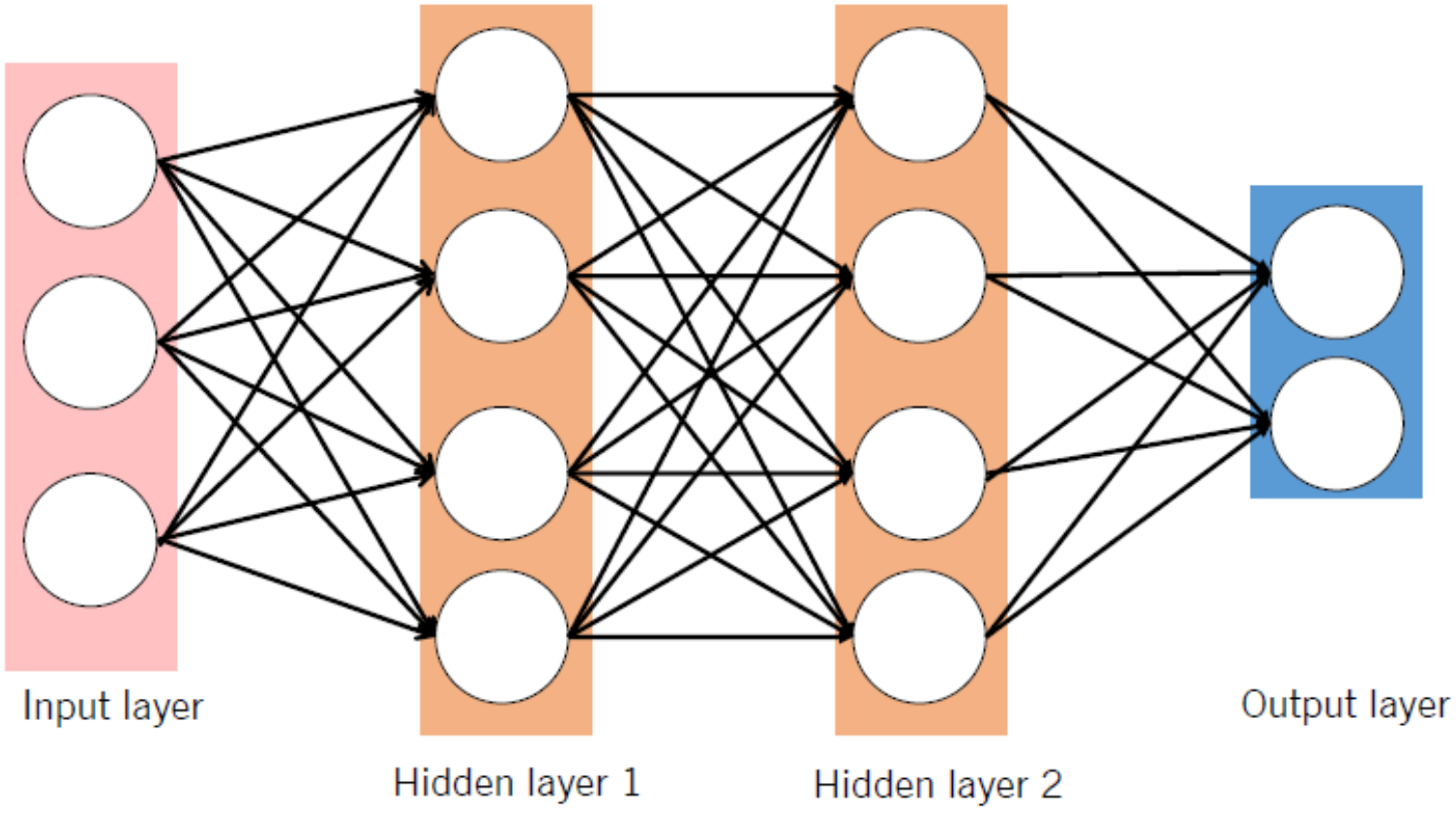
# Artificial Intelligence (AI)



# AGENDA

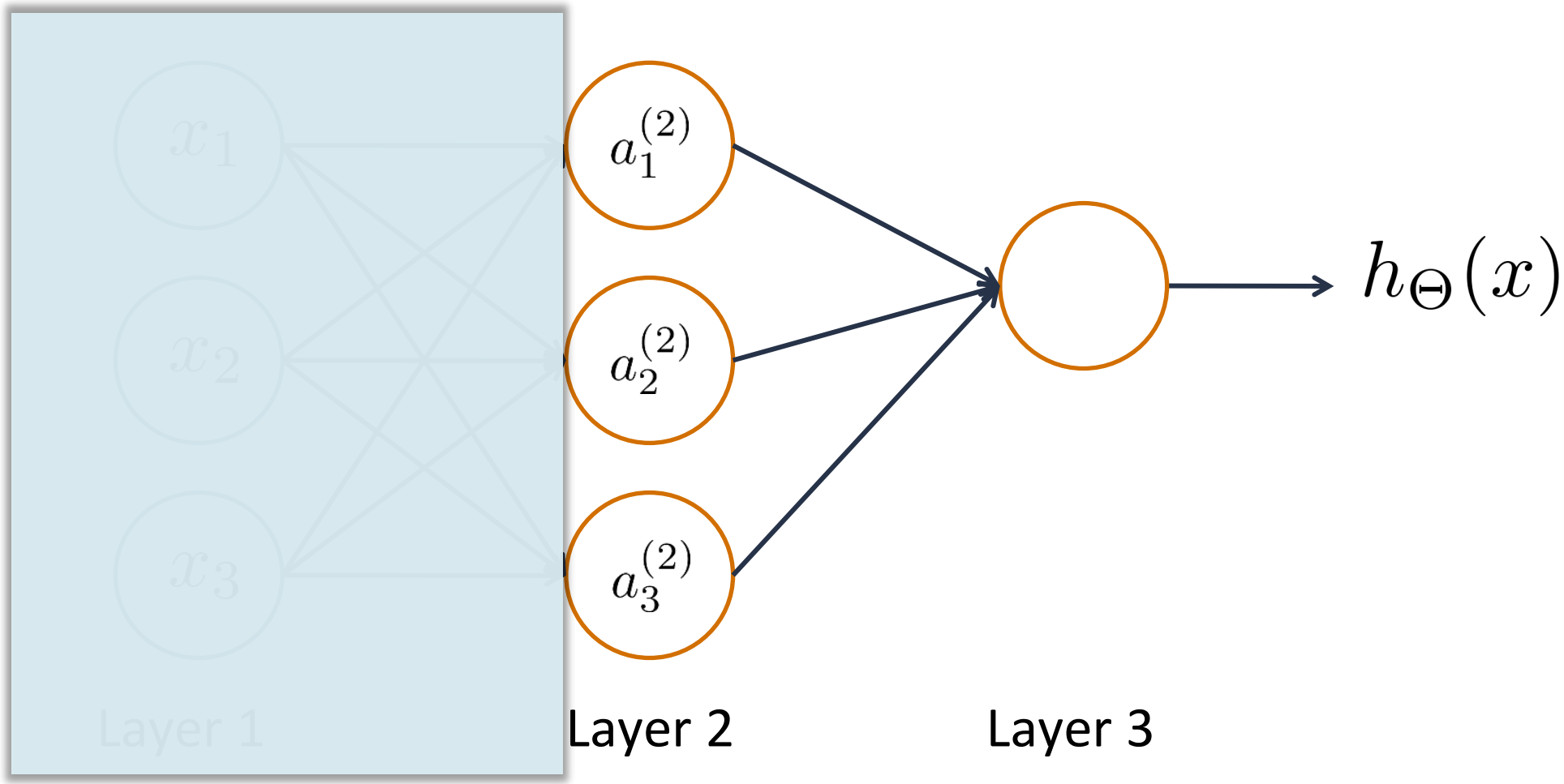
- Feed Forward Neural Network
- Backpropagation
- Activation Functions
- Loss functions
- Optimizers

# Feedforward Neural Network

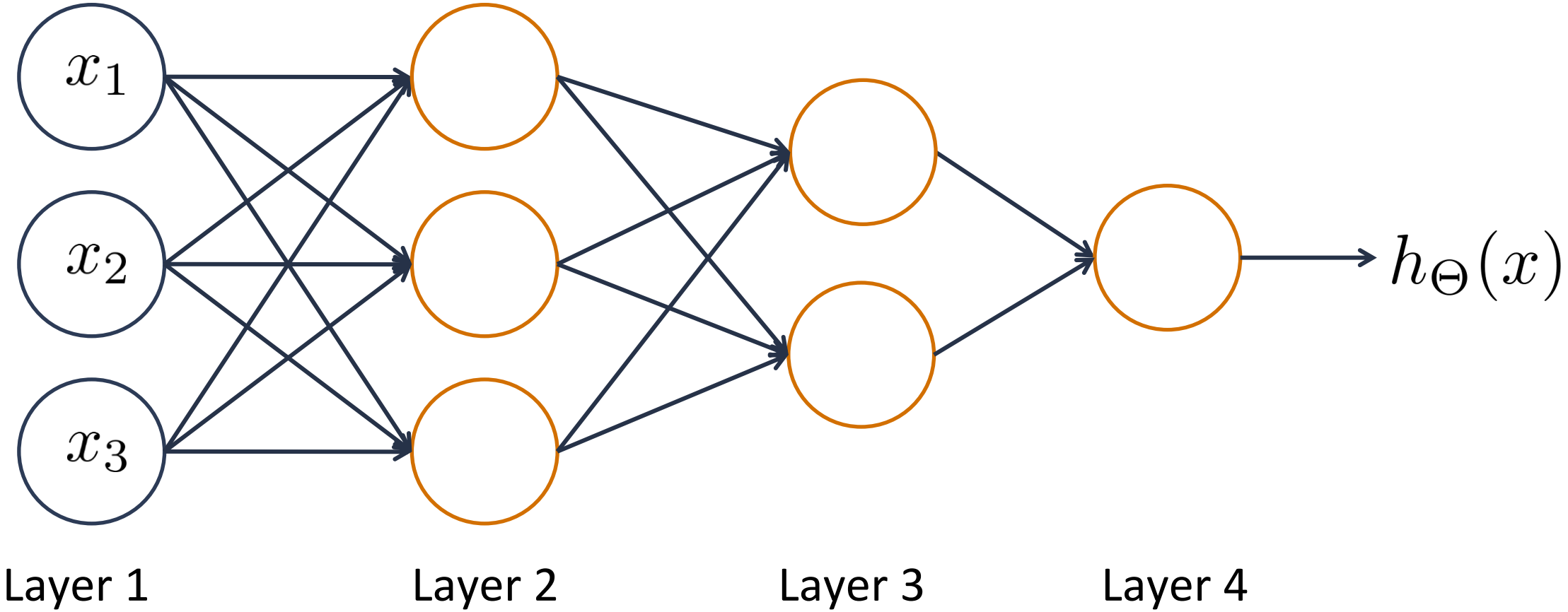



Back to main

# Neural Network learning its own features



# Other network architectures



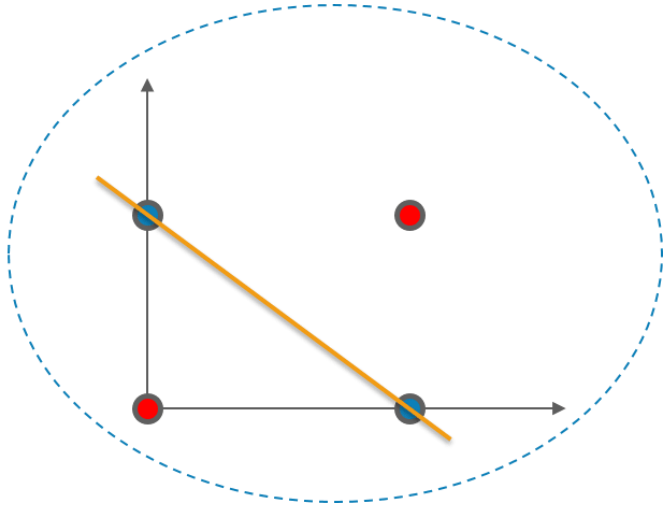
# Limitation: Problem_1

- **Single-Layer Perceptron's cannot classify non-linearly separable data points**

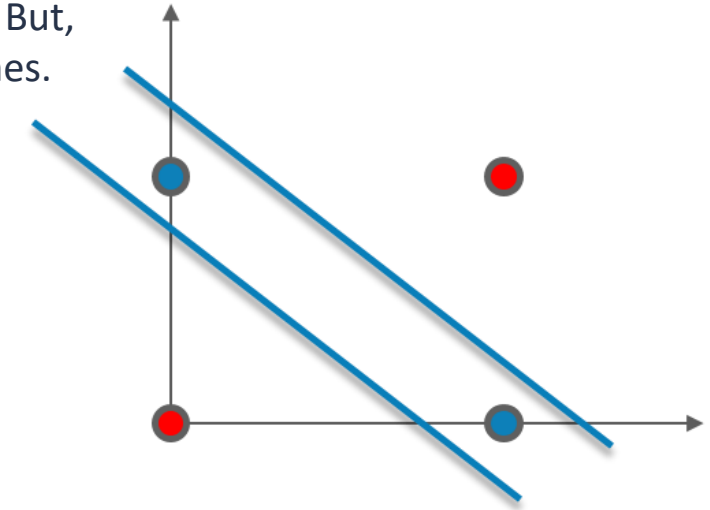
XOR



| X1 | X2 | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |



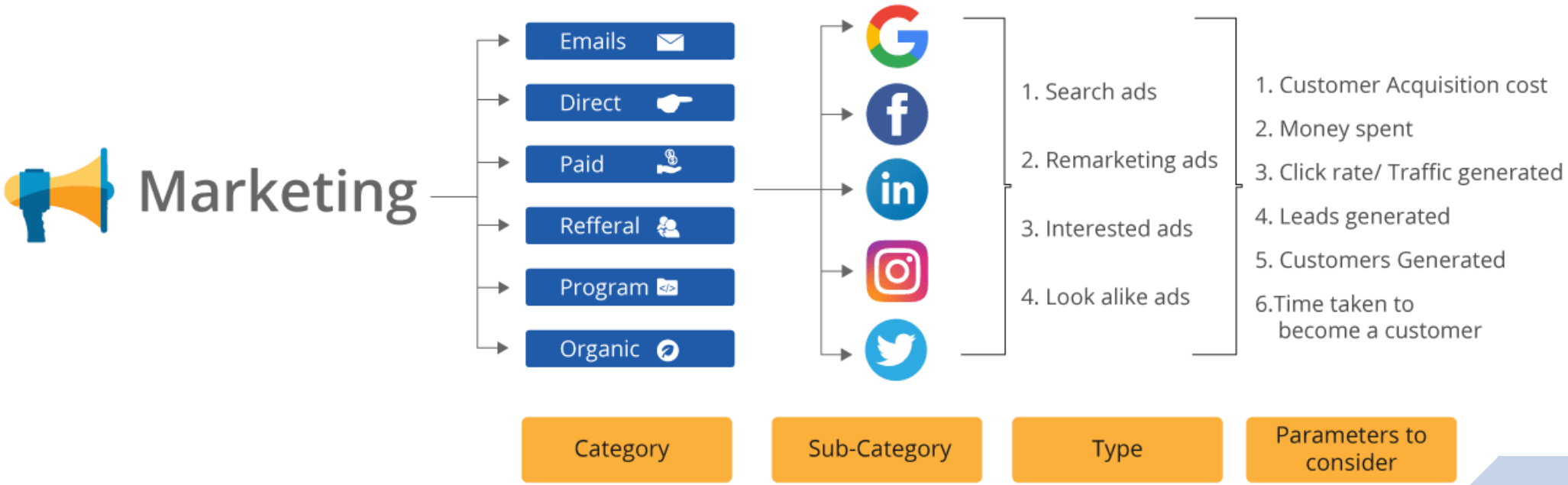
- Here, you cannot separate the high and low points with a single straight line. But, we can separate it by two straight lines. Consider the diagram below:



# Limitation: Problem_2

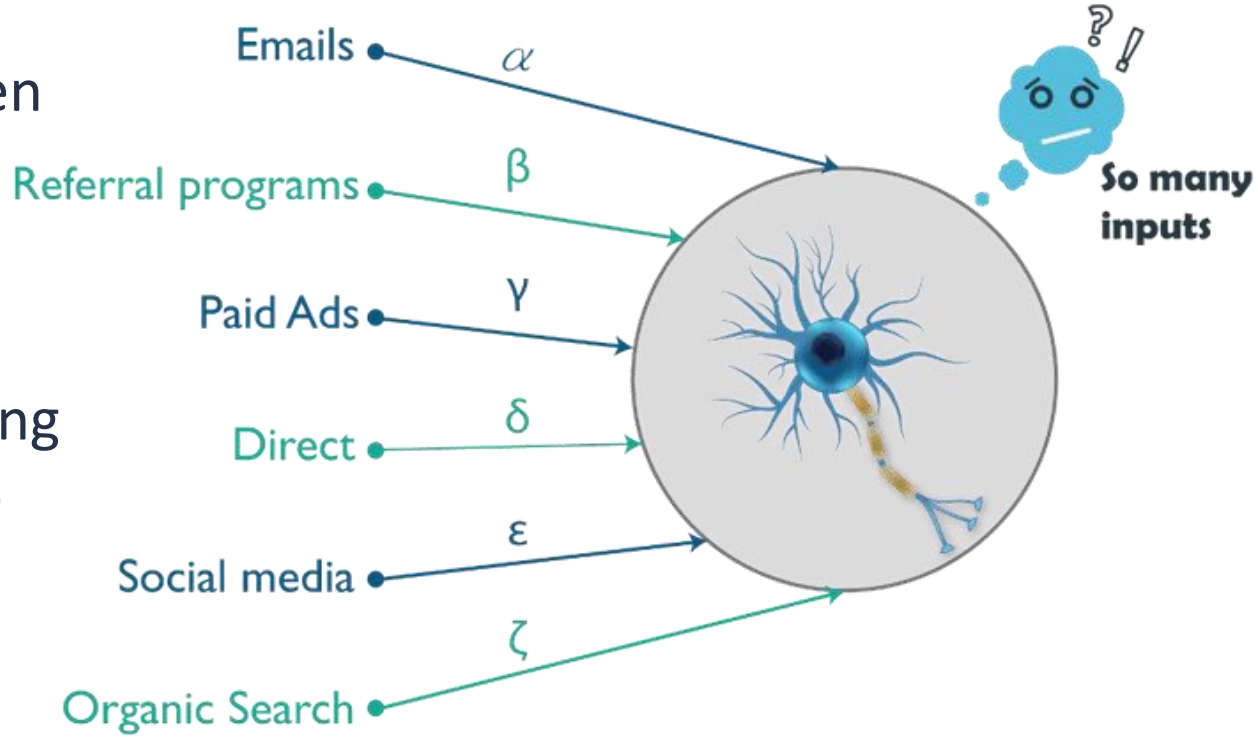
**Complex problems, that involve a lot of parameters cannot be solved by Single-Layer Perceptrons:**

Example: As an E-commerce firm, you have noticed a decline in your sales.



# Limitation: Problem_2

- Number of sales that would happen would be dependent on different categorical inputs, their sub categories and their parameters.
- However, computing and calculating from so many inputs and their sub parameters is not possible just through one neuron (Perceptron).




Back to main

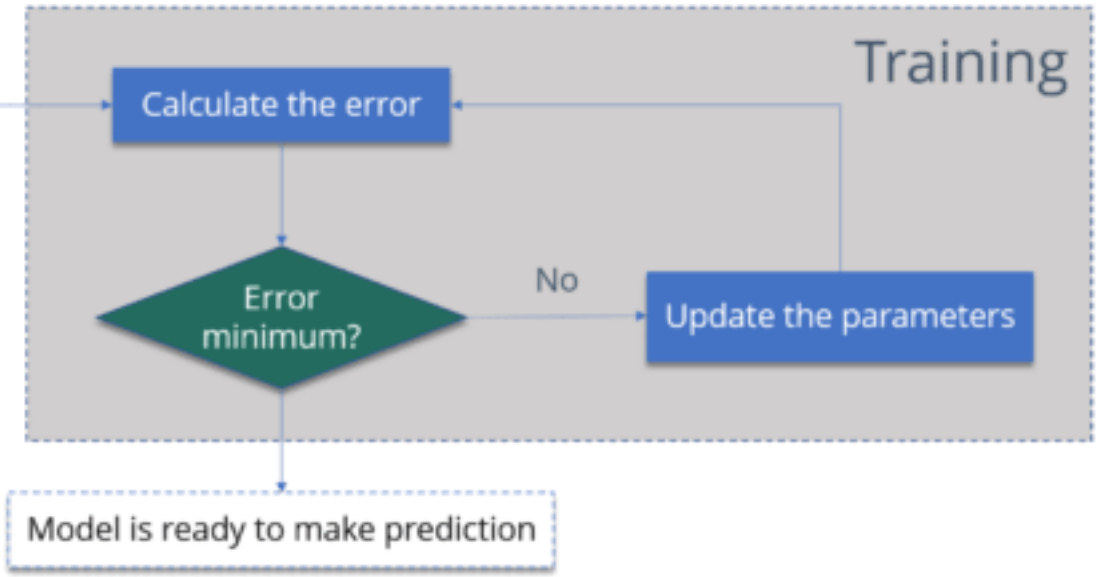
# Backpropagation

Back to main

# Why We Need Backpropagation?

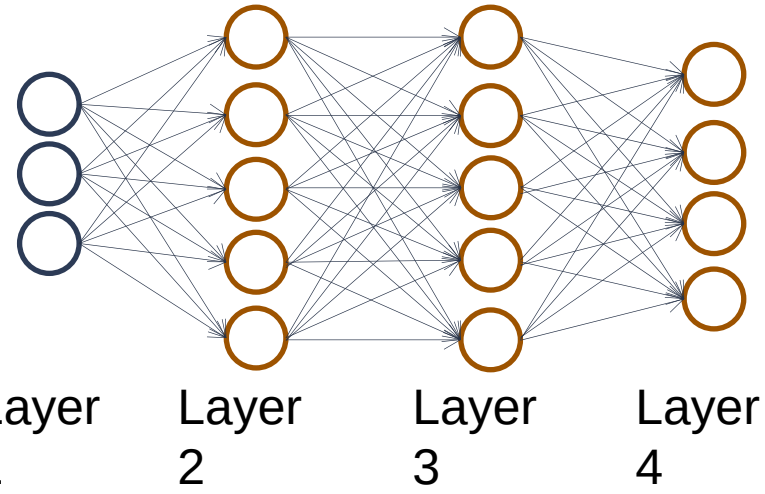
- Let me summarize the steps for you:
  - Calculate the error
  - Minimum Error
  - Update the parameters
  - Model is ready to make a prediction



# Gradient computation: Backpropagation algorithm

Intuition: $\delta_j^{(l)} =$ “error” of node $j$ in layer $l$ .




For each output unit (layer L = 4)

$$\delta_j^{(4)} = a_j^{(4)} - y_j$$

$$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} .* g'(z^{(3)})$$

$$\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} .* g'(z^{(2)})$$

# What is backpropagation doing?

$$J(\Theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log(h_\Theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - (h_\Theta(x^{(i)})))\right]$$
$$+\frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}(\Theta_{ji}^{(l)})^2$$

Focusing on a single example $x^{(i)}$, $y^{(i)}$, the case of 1 output unit, and ignoring regularization ($\lambda = 0$),

$$\text{cost(i)} = y^{(i)} \log h_\Theta(x^{(i)}) + (1 - y^{(i)}) \log h_\Theta(x^{(i)})$$

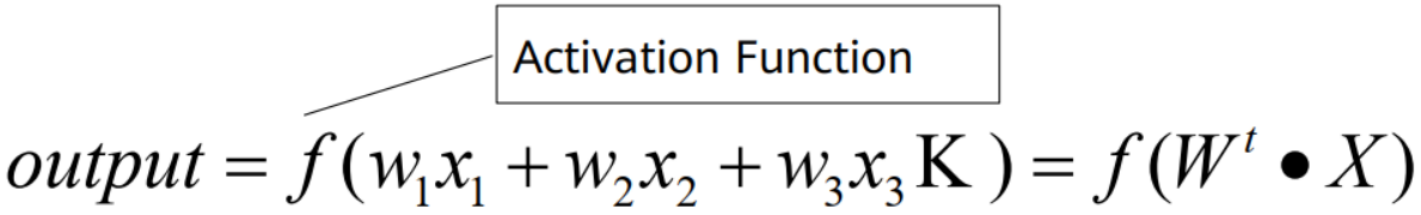
(Think of $\text{cost(i)} \approx (h_\Theta(x^{(i)}) - y^{(i)})^2$)

I.e. how well is the network doing on example i?

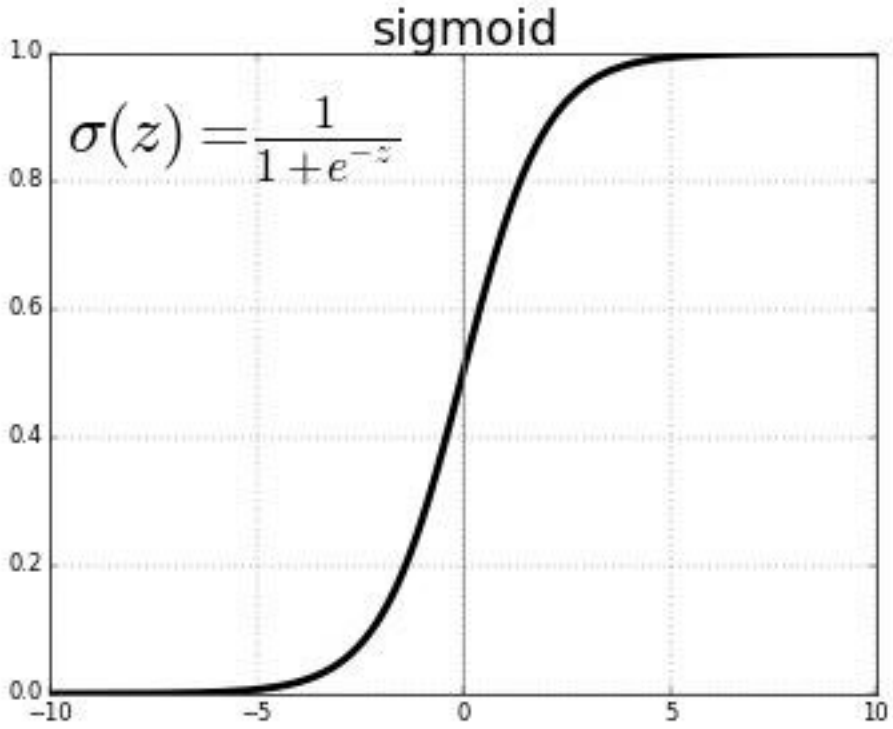
Back to main

# Activation Functions

Activation functions are important for the neural network model to learn and understand complex non-linear functions. They allow introduction of non-linear features to the network

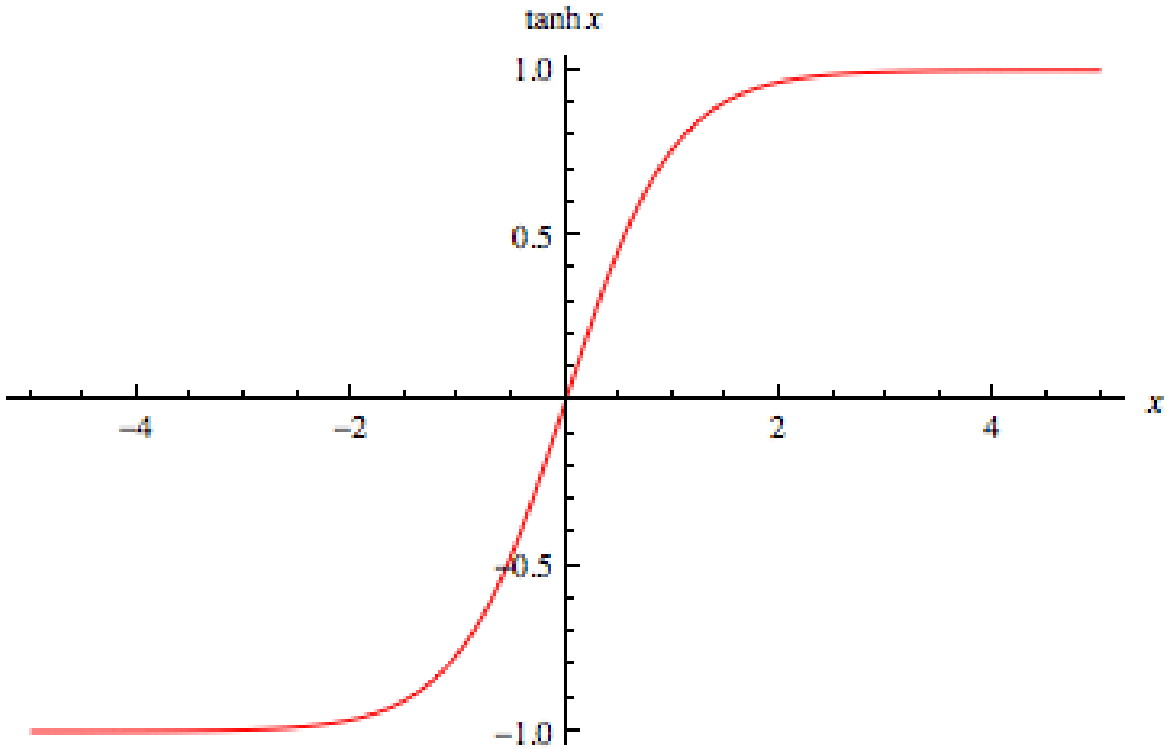


# Sigmoid Function



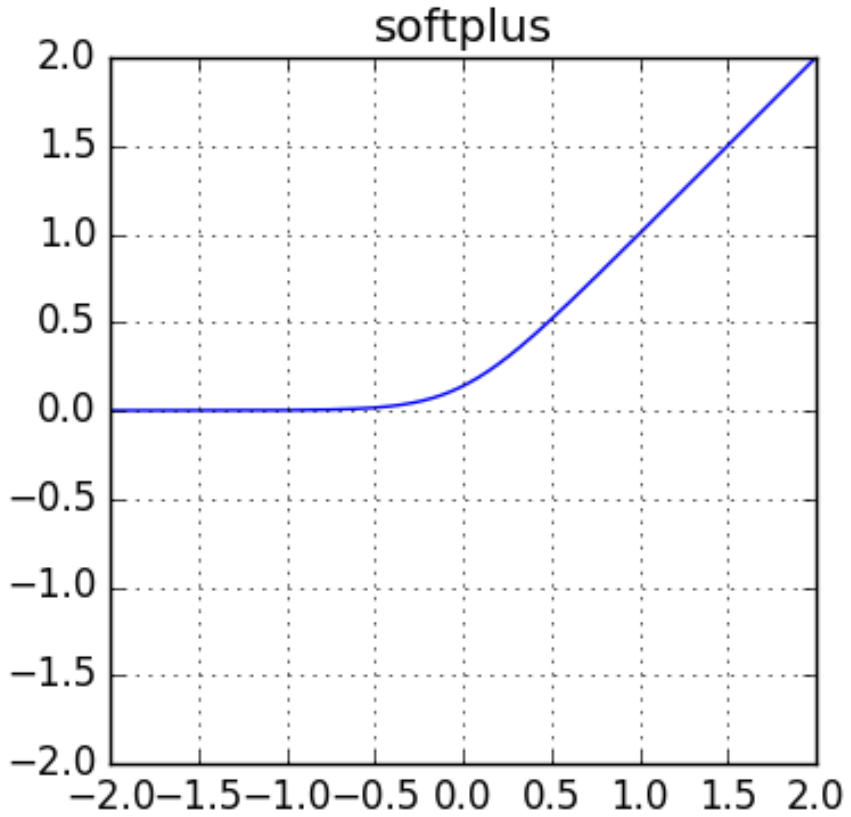
# Tanh Function

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

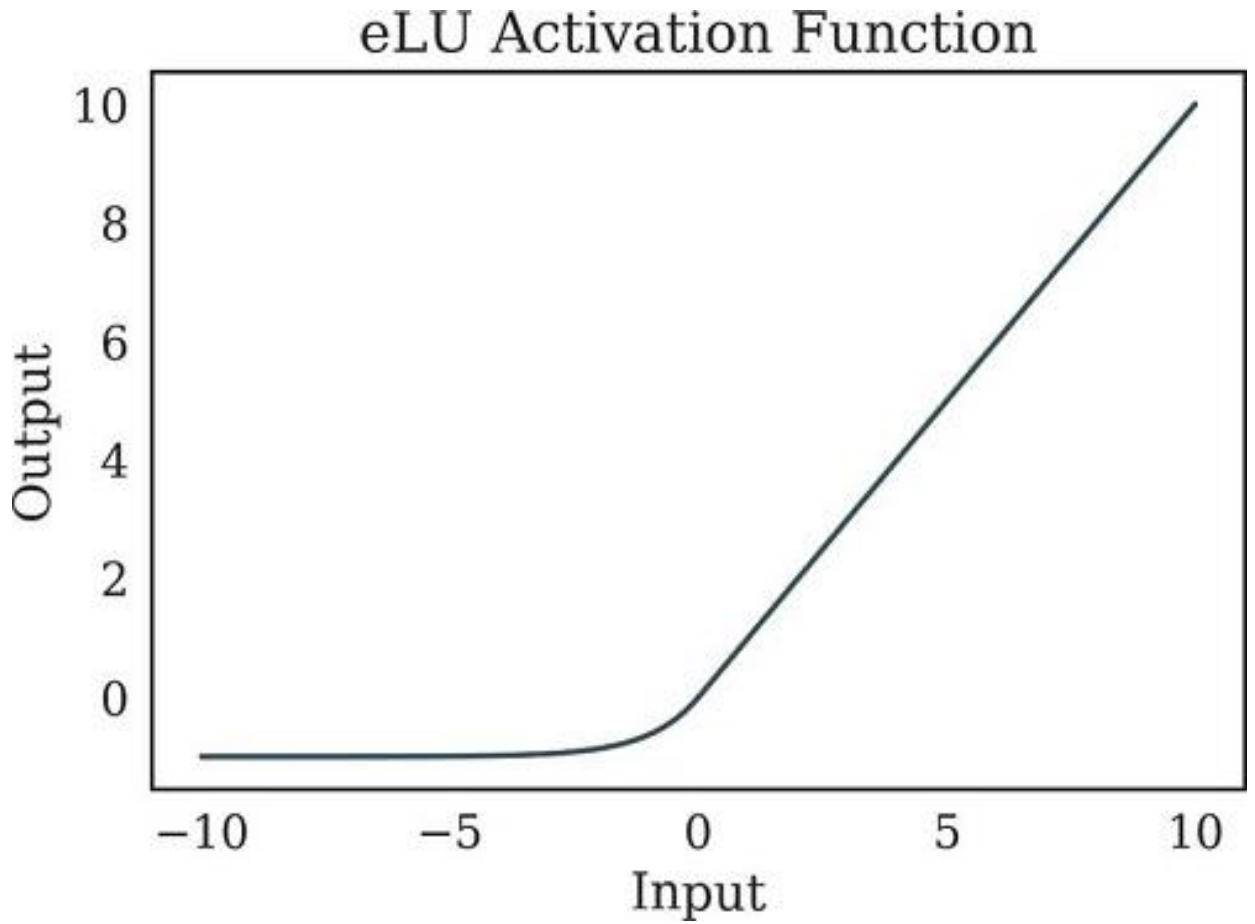


# Softplus function

$$f(x) = \ln(e^x + 1)$$



# ReLU Function




Back to main

# Loss functions

Back to main

# Loss functions

- Loss function is a mathematical function that calculates the difference between the predicted output of a model and the actual output. Common loss functions are:
  - Mean Squared Error (MSE): For regression problem
  - Mean Absolute Error (MAE): For regression problem
  - Binary Cross-Entropy: For binary classification problem
  - Categorical Cross-Entropy: For multi classification problem
  - Sparse Categorical Cross-Entropy: For multi classification problem when the labels are integer.

Back to main

# Optimizers

Back to main

# Optimizers

- In deep learning, an optimizer is an algorithm that adjusts the weights and biases of a neural network during training to minimize the loss function.
- Common optimizers: common GD optimizer, momentum optimizer, Nesterov, AdaGrad, AdaDelta, RMSProp, Adam, AdaMax, and Nadam